校园活动运营管理平台初始开发文档（V1.0）

1. 文档目标

本文档用于把原有“短链平台”能力升级为一个可实际开发、可部署、可演示、可逐步上线的 **校园活动运营管理平台**。

本版本重点解决三件事： 1. 明确产品定位与 MVP 范围 2. 设计适合 AI / vibe coding 的模块边界 3. 从一开始就把 **埋点统计** 和 **2C4G 云服务器上线约束** 纳入设计

---

2. 产品定位

2.1 产品名称（暂定）

CampusFlow / 校园活动运营管理平台

2.2 核心定位

面向学院、社团、学生组织、班委等活动发布者，提供： - 活动创建与发布 - 活动报名与签到 - 短链与二维码分发 - 渠道访问与转化统计 - 活动复盘分析

2.3 核心闭环

创建活动 → 生成短链/二维码 → 多渠道传播 → 学生访问 → 报名/签到 → 数据统计 → 活动复盘

2.4 为什么不是纯活动报名系统

本平台的差异化不只是“活动 CRUD”，而是把你已有的 **短链生成、访问统计、分渠道分析** 能力内化为平台基础设施，形成“活动运营闭环”。

---

3. 第一阶段产品目标

3.1 第一版目标

在 2C4G 云服务器上可以稳定部署一个 **可演示、可真实使用的小规模版本**。

第一版要保证： - 发布者能登录系统并创建活动 - 系统能生成活动短链和二维码 - 学生能访问活动详情并报名 - 发布者能查看报名名单和基础数据 - 平台能记录访问、点击、报名、签到等核心埋点 - 可以在单机服务器上稳定运行

3.2 第一版不强求实现

以下能力先预留接口或设计，不作为第一版必须完成： - 微服务拆分 - 分库分表 - RabbitMQ 完整异步架构 - 复杂审批流 - 多租户高级权限模型 - 大规模消息推送 - 超复杂 BI 大屏

---

4. 目标用户

4.1 活动发布者

如学院老师、辅导员、学生会负责人、社团负责人、班委。

核心诉求： - 快速创建活动 - 方便传播活动 - 统计活动效果 - 查看报名与签到情况

4.2 学生用户

通过链接或二维码访问活动详情页，完成报名与签到。

4.3 平台管理员

管理用户、组织、活动与平台整体数据。

---

5. 平台功能结构

5.1 主业务模块

活动中心

- 创建活动
- 编辑活动
- 删除活动
- 发布/下线活动
- 设置报名时间、活动时间、人数上限、地点、封面、说明

5.2 核心支撑模块

A. 报名与签到中心

- 学生报名
- 报名去重
- 报名人数统计
- 签到码/签到页
- 签到记录管理

B. 短链与二维码中心

- 为活动自动生成短链
- 不同传播渠道生成不同短链
- 自动生成二维码
- 设置短链有效期
- 短链启用/停用

C. 数据分析中心

- 活动访问量

- 报名量
- 签到量
- 转化率
- 渠道访问对比
- 时间趋势分析

D. 用户与组织中心

- 登录注册
- 角色管理
- 学院/社团/班级分组
- 组织内活动归属管理

---

6. MVP 功能范围

6.1 第一版必做功能

账号体系

- 用户登录
- 用户退出
- JWT 鉴权
- 发布者身份区分

活动管理

- 创建活动
- 编辑活动
- 发布活动
- 下线活动
- 查看活动列表
- 活动详情页

报名管理

- 学生填写报名表
- 报名名单查询
- 限制重复报名
- 导出报名名单（第二优先，可延后）

签到管理

- 活动签到
- 展示签到状态
- 查看签到名单

短链能力

- 自动生成活动短链
- 支持渠道化短链
- 生成二维码

- 访问短链自动跳转到活动页

数据统计

- 活动总访问量
- 活动报名量
- 活动签到量
- 渠道访问量
- 最近 7 天趋势

平台基础管理

- 用户列表
- 活动列表
- 平台概览页

6.2 第二阶段增强功能

- 审核流
- 活动收藏/分享
- 批量导入活动参与名单
- 二维码签到核验
- 活动海报生成
- 邮件/短信/站内消息提醒
- 流量包或额度系统
- 高级渠道分析
- 风险控制与反刷量

---

7. 页面规划

7.1 发布者后台

1. 登录页
2. 工作台首页
3. 活动列表页
4. 新建活动页
5. 编辑活动页
6. 活动详情管理页
7. 报名管理页
8. 签到管理页
9. 数据分析页
10. 个人中心页

7.2 学生侧页面

1. 活动详情页
2. 活动报名页
3. 报名成功页

4. 活动签到页
5. 活动已结束页
6. 活动未开始/已下线页

7.3 管理端页面

1. 平台概览页
2. 用户管理页
3. 组织管理页
4. 活动总表页
5. 数据总览页

---

8. 核心业务流程

8.1 创建活动流程

1. 发布者填写活动基本信息
2. 保存活动数据
3. 系统自动生成活动访问链接
4. 系统生成默认短链
5. 发布者可继续生成不同渠道短链
6. 发布成功后用于海报、班群、公众号等传播

8.2 访问活动流程

1. 学生访问短链或扫码
2. 系统记录访问埋点
3. 跳转活动详情页
4. 若活动有效则可报名
5. 若活动失效则展示失效页

8.3 报名流程

1. 学生填写报名信息
2. 系统校验报名时间与人数上限
3. 校验是否已报名
4. 保存报名记录
5. 记录报名转化埋点
6. 返回报名成功页面

8.4 签到流程

1. 学生进入签到页或出示签到码
2. 系统校验活动状态
3. 校验是否已报名/是否重复签到
4. 保存签到记录
5. 更新活动统计

---

9. 埋点设计（第一版重点）

这一部分是本项目的核心竞争力之一。第一版就要把埋点设计好，不然后面统计会很乱。

9.1 埋点目标

埋点不是为了“多记日志”，而是为了回答这些问题： - 某活动被多少人看过？ - 哪个渠道带来的访问最多？ - 哪个渠道带来的报名转化更高？ - 活动报名高峰在什么时间？ - 最终有多少人报名、多少人签到？

9.2 埋点事件列表

A. 页面访问类

- `activity_view`：访问活动详情页
- `activity_share_open`：从分享链接进入活动页
- `short_link_visit`：访问短链

B. 报名类

- `activity_register_submit`：提交报名
- `activity_register_success`：报名成功
- `activity_register_fail`：报名失败

C. 签到类

- `activity_checkin_submit`：提交签到
- `activity_checkin_success`：签到成功
- `activity_checkin_fail`：签到失败

D. 发布者操作类

- `activity_create`
- `activity_update`
- `activity_publish`
- `channel_link_create`
- `activity_disable`

9.3 埋点字段建议

每条事件建议包含： - event_id - event_type - event_time - user_id（可为空） - activity_id - short_code（可为空） - channel_code（如班群/公众号/海报） - ip - user_agent - referer - device_type - os - browser - request_id - extra_json

9.4 第一版埋点实现建议

为了适应 2C4G 单机环境，第一版不建议上太重的埋点架构，建议这样做：

方案

- 访问类埋点：先写数据库 + 部分热数据写 Redis

- 聚合统计：通过定时任务定期汇总
- 页面趋势：直接查汇总表，而不是每次扫明细表

不建议第一版就做

- Kafka 大数据链路
- 实时流式计算
- ELK 全量日志分析

9.5 埋点展示指标

第一版页面上展示： - 活动浏览量 PV - 活动报名人数 - 活动签到人数 - 报名转化率 = 报名人数 / 访问人数 - 签到转化率 = 签到人数 / 报名人数 - 渠道访问排行 - 最近 7 天访问趋势

---

10. 技术架构建议

10.1 为什么第一版不直接做微服务

你目前有 2C4G 服务器，而且项目目标是尽快做出可上线版本。

因此第一版建议： **前后端分离 + 单体后端 + Redis + MySQL + Nginx + Docker Compose**

这套架构有几个好处： - 部署简单 - 资源占用低 - 便于开发和调试 - 后续能平滑升级到微服务

10.2 推荐技术栈

前端

- React
- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- ECharts

后端

- Java 17
- Spring Boot 3
- Spring Security
- JWT
- MyBatis-Plus
- MySQL 8
- Redis
- Lombok
- Knife4j / Swagger

运维与部署

- Docker
- Docker Compose
- Nginx
- Spring Boot Actuator

---

11. 适配 2C4G 服务器的部署设计

11.1 部署原则

你这台 2C4G 服务器第一版够用，但前提是： - 服务数量不能太多 - JVM 不能开太大 - 不要同时跑一堆重型中间件 - 统计类查询优先查汇总表 - 图片、文件不要长期堆在本机

11.2 建议部署组件

第一版建议部署以下组件： 1. Nginx 2. 前端应用（Next.js 静态部署或 Node 服务） 3. Spring Boot 后端 4. MySQL 5. Redis

11.3 第一版不建议在同机部署的重型组件

- Elasticsearch
- Kafka
- 多个 Java 微服务
- Prometheus + Grafana 全家桶（可后加）
- RabbitMQ（不是不能上，而是第一版未必要上）

11.4 资源分配建议

大致思路

- MySQL：较保守内存占用
- Redis：控制 maxmemory
- Java 后端：限制 Xms/Xmx
- 前端：尽量静态化或轻量运行

建议

- Java 应用：`-Xms512m -Xmx1024m`
- Redis：控制在 256MB ~ 512MB
- MySQL：适度调小 buffer
- 尽量避免多个 Node + Java 进程同时暴涨

11.5 上线初期流量定位

第一版按“小规模校园内部试运行”考虑： - 活动发布者数量有限 - 同时在线人数中低 - 峰值集中在活动宣传时段

因此单机完全可以先支撑。

---

12. 模块划分

12.1 用户模块

- 登录
- 用户信息
- 角色管理
- 组织归属

12.2 活动模块

- 活动基础信息
- 活动状态管理
- 活动详情展示

12.3 报名模块

- 报名表单
- 报名校验
- 报名列表查询

12.4 签到模块

- 签到提交
- 签到状态管理
- 签到名单查询

12.5 短链模块

- 默认活动短链
- 渠道短链
- 短码生成
- 二维码生成
- 短链访问

12.6 埋点统计模块

- 行为事件上报
- 事件存储
- 汇总统计
- 趋势分析

12.7 管理后台模块

- 平台概览
- 用户管理

- 活动管理

---

13. 数据库设计（初稿）

13.1 用户表 `user`

字段建议： - id - username - password_hash - nickname - email - phone - role - org_id - status - created_at - updated_at

13.2 组织表 `organization`

字段建议： - id - name - type（学院/社团/班级） - parent_id - created_at - updated_at

13.3 活动表 `activity`

字段建议： - id - org_id - creator_id - title - summary - cover_url - location - content - status - register_start_at - register_end_at - event_start_at - event_end_at - capacity - view_count - register_count - checkin_count - created_at - updated_at

13.4 活动渠道表 `activity_channel`

字段建议： - id - activity_id - channel_code - channel_name - short_code - short_url - qr_code_url - status - expire_at - created_at

13.5 报名表 `activity_registration`

字段建议： - id - activity_id - user_name - student_no - phone - class_name - extra_form_json - status - registered_at

13.6 签到表 `activity_checkin`

字段建议： - id - activity_id - registration_id - checkin_method - checked_in_at - status

13.7 埋点事件表 `tracking_event`

字段建议： - id - event_type - activity_id - channel_code - short_code - user_id - ip - user_agent - referer - device_type - os - browser - request_id - extra_json - event_time

13.8 统计汇总表 `activity_stats_daily`

字段建议： - id - activity_id - stat_date - view_count - register_count - checkin_count - created_at - updated_at

13.9 渠道统计表 `activity_channel_stats_daily`

字段建议： - id - activity_id - channel_code - stat_date - view_count - register_count - checkin_count - created_at - updated_at

---

14. 接口设计（初稿）

14.1 认证相关

- `POST /api/auth/login`
- `GET /api/auth/me`
- `POST /api/auth/logout`

14.2 活动相关

- `POST /api/activities`
- `GET /api/activities`
- `GET /api/activities/{id}`
- `PUT /api/activities/{id}`
- `POST /api/activities/{id}/publish`
- `POST /api/activities/{id}/offline`

14.3 报名相关

- `POST /api/activities/{id}/register`
- `GET /api/activities/{id}/registrations`
- `GET /api/activities/{id}/registration-stats`

14.4 签到相关

- `POST /api/activities/{id}/checkin`
- `GET /api/activities/{id}/checkins`

14.5 渠道短链相关

- `POST /api/activities/{id}/channels`
- `GET /api/activities/{id}/channels`
- `POST /api/channels/{channelId}/disable`

14.6 数据统计相关

- `GET /api/stats/activities/{id}/overview`
- `GET /api/stats/activities/{id}/trend`
- `GET /api/stats/activities/{id}/channels`

14.7 埋点相关

- `POST /api/tracking/events`

14.8 对外访问

- `GET /s/{shortCode}`
- `GET /activity/{id}`

---

15. 短链设计建议

15.1 第一版策略

第一版不建议一开始就引入最复杂的 MurmurHash32 + Base62 + 异步重试链路。

建议优先采用： - 随机 Base62 短码 - 冲突检测 - 冲突后重试

15.2 为什么这样设计

- AI 更容易稳定生成代码
- 单机场景下完全够用
- 便于快速上线
- 后续可以平滑替换成更接近原项目的策略

15.3 第二阶段升级方向

- 引入更稳定的短码生成策略
- 引入 Redis 缓存热点短链
- 引入渠道短链统计聚合
- 再逐步靠近你原来的工程化设计

---

16. 缓存设计

16.1 需要缓存的内容

- 短链映射关系
- 热门活动详情
- 活动统计概览
- 登录态/鉴权相关信息

16.2 Key 建议

- `shortlink:code:{code}`
- `activity:detail:{id}`
- `activity:stats:{id}`
- `activity:channel:stats:{id}`

16.3 缓存使用原则

- 活动详情可短期缓存

- 热门活动优先缓存
- 统计优先查汇总表，再配合 Redis 缓存
- 对不存在短码可做空值缓存防穿透

---

17. 定时任务设计

第一版可先用 Spring Schedule。

需要的任务

- 活动状态定时变更
- 每日统计汇总任务
- 失效短链清理/状态同步
- 过期活动缓存清理

第二阶段再考虑 XXL-Job。

---

18. 安全与风控

第一版建议做这些基础能力： - 登录接口限流 - 报名接口限流 - URL 参数校验 - 防止重复报名 - 活动状态校验 - 基础操作日志 - 后台接口权限控制

后续增强： - IP 维度频控 - 渠道刷量识别 - 黑名单机制 - 签到风控

---

19. 开发阶段规划

Milestone 1：项目骨架搭建

- 前端基础布局
- 后端 Spring Boot 工程
- 用户表、活动表、报名表、埋点表初始化
- 登录接口

Milestone 2：打通主链路

- 创建活动
- 活动详情展示
- 自动生成短链
- 学生访问活动页
- 学生报名

Milestone 3：加入埋点统计

- 访问埋点
- 报名埋点
- 汇总表

- 后台统计页

Milestone 4：加入签到与二维码

- 签到页
- 签到管理
- 二维码生成

Milestone 5：优化上线能力

- Redis 缓存
- 定时任务
- Docker Compose 部署
- Nginx 反向代理
- 简单监控与日志

---

20. 上线建议（适合你的 2C4G 服务器）

第一版上线策略

- 先灰度给少量同学/组织试用
- 图片尽量走对象存储或图床
- 数据库定期备份
- 访问日志和埋点明细定期清理/归档
- 优先保证主链路稳定：访问、报名、签到

部署方式

推荐使用 Docker Compose 管理： - nginx - app-backend - mysql - redis - frontend

监控建议

第一版可先做轻量监控： - 应用日志文件 - Nginx 访问日志 - Spring Boot Actuator - 简单错误告警（后加）

---

21. 当前最适合继续细化的内容

下一轮最建议继续补这 4 项： 1. 页面原型结构 2. 数据库字段详细定义 3. 接口返回格式统一规范 4. Docker Compose 上线配置方案

---

22. 结论

这版平台不是简单把“短链系统”包一层页面，而是把你的短链能力升级成校园活动运营基础设施：

- 活动是主业务
- 短链是传播工具
- 埋点是数据底座

- 统计是平台亮点
- 2C4G 服务器可以作为第一版上线环境

整体开发策略是： **先做单体可上线版，再逐步演进成更强的工程化版本。**